2 Définitions

L'ensemble des n tâches (tasks, jobs) est noté $T = \{T_1, T_2, \ldots, T_n\}$.

L'ensemble des m machines (machines, processors) est noté $M = \{M_1, M_2, \ldots, M_m\}$.

2.1 Caractérisation des machines

il existe deux grandes classes de machines :

- Les machines parallèles

- Les machines spécialisées

Les machines parallèles (parallel machines) : Ces machines exécutent les mêmes traitements et sont donc capables de traiter n'importe quelle tâche. En fonction de la vitesse de traitement, nous distinguons trois types de machines parallèles :

— Les machines identiques (identical machines) : Pour ces machines, les vitesses de traitement des tâches sont toutes égales.

— Les machines uniformes (uniform machines) : Pour ces machines, les vitesses de traitement des tâches sont différentes mais que la vitesse de chaque machine est constante et ne dépend pas des tâches de T.

— Les machines générales (unrelated machines) : Pour ces machines, la vitesse de traitement d'une machine dépend de la tâche qui est traitée.

Les machines spécialisées ou dédiées (dedicated machines) : Ces machines sont spécialisées à l'exécution de certaines tâches. Dans le cas de machines spécialisées, nous distinguons trois types de configuration des machines ou modes de traitement :

— Ateliers à cheminement unique ou ateliers en ligne (Flow shop) : les tâches de T doivent être traitées par toutes les machines dans un même ordre. Le même ordre de traitement pour toutes les tâches.

— Ateliers à cheminement libre (Open shop) : les tâches de T doivent être traitées par toutes les machines mais aucun ordre de traitement n'est imposé pour les tâches.

— Ateliers à cheminements multiples (Job shop) : le sous-ensemble de machines devant traiter une tâche et l'ordre du traitement sont arbitraires, mais doivent être spécifiés à l'avance.

2.2 Description des tâches

Dans le cas de machines parallèles, chaque tâche peut être traitée par n'importe quelle machine.

Si les machines sont spécialisées, chaque tâche $T_i \in T$ est divisée en un nombre fini d'opérations $O_{i1}, O_{i2}, \ldots, O_{ik_i}$. Chacune pouvant nécessiter une machine différente (k_i étant le nombre d'opérations de la tâche T_i) :

Chaque tâche $T_i \in T$ peut être caractérisée par les données suivantes :

1. Un vecteur temps de traitement (processing time) $\overline{p_i} = \{p_{i1}, \ldots, p_{im}\}$: p_{ij} est le temps de traitement nécessaire à la machine M_j pour terminer le traitement de la tâche T_i. (durée de traitement)

   — Dans le cas de machines identiques, nous avons : $p_{ij} = p_i$ pour $j = 1, \ldots, m$.
   — Dans le cas de machines uniformes, nous avons : $p_{ij} = p_i/b_j$ pour $j = 1, \ldots, m$ où b_j est le facteur vitesse de traitement de la machine M_j et p_i un temps de traitement standard (correspondant à $b_j = 1$)).

2. Une date d'arrivée, de disponibilité ou de début au plus tôt (release date) r_i : c'est la date à laquelle la tâche T_i est prête à être traitée. Si pour toutes les tâches de T, les dates d'arrivée sont toutes égales, on pose $r_i = 0$ pour $i = 1, \ldots, n$.

3. Une date échue ou de fin au plus tard (due date) d_i : si le traitement de la tâche T_i doit être achevé avant la date d_i.

4. Une date limite ou absolue (deadline) $\tilde{d}_i$: si le traitement de la tâche T_i doit obligatoirement être achevé avant la date $\tilde{d}_i$.

5. Un poids (weight) w_i : qui exprime une priorité relative (une pénalité, un profit, etc.) de la tâche T_i.

Donnons maintenant quelques définitions concernant les préemptions et les contraintes de précédence entre les tâches.

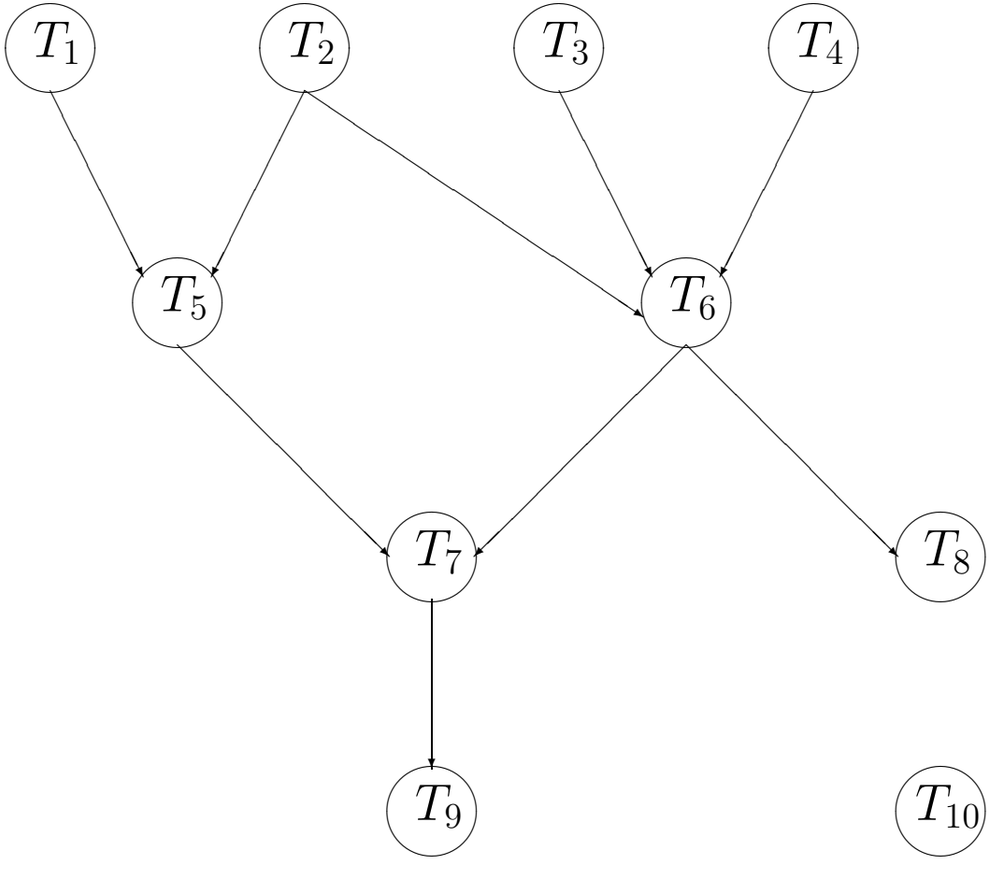
Définition 1 *Un mode de traitement est dit préemptif ou avec préemption (preemptive) si chaque tâche (ou opération dans le cas de machines spécialisées) peut être arrêtée à tout instant pour terminer son exécution plus tard sans aucun coût, peut être sur une autre machine (on parlera aussi de tâches morcelables ou interruptibles). Si ce n'est pas la cas, on dira que le mode de traitement est non préemptif ou sans préemption (nonpreemptive).*

Définition 2 *Dans l'ensemble T, nous pouvons établir des contraintes de précédence (precedence constraints) entre les tâches :*
$T_i < T_j$ veut dire que le traitement de la tâche T_i doit être achevé avant que celui de T_j ne commence. L'ensemble T des tâches est ainsi ordonné par la relation $<$. Si au moins deux tâches de T sont comparables, on dit que les tâches de T sont dépendantes. Sinon les tâches de T sont dites indépendantes.

Un ensemble de tâches ordonné par une relation de précédence est habituellement représenté comme un graphe orienté dans lequel les sommets représentent les tâches et les arcs les contraintes de précédence.

Considérons l'instance ci-dessous avec 10 tâches et 3 machines identiques où la préemption n'est pas autorisée. Les temps de traitement, les dates de disponibilité et les dates échues sont données dans la table ci-dessous. La figure représente les contraintes de précédence.

T_i	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9	T_{10}
p_i	2	3	1	2	4	1	2	6	2	1
r_i	0	0	0	0	3	3	3	4	5	0
d_i	3	4	3	2	7	5	6	10	8	11



2.3 Ordonnancement

Un ordonnancement est l'allocation des machines de M aux tâches de T en un temps tel que les conditions suivantes soient satisfaites :

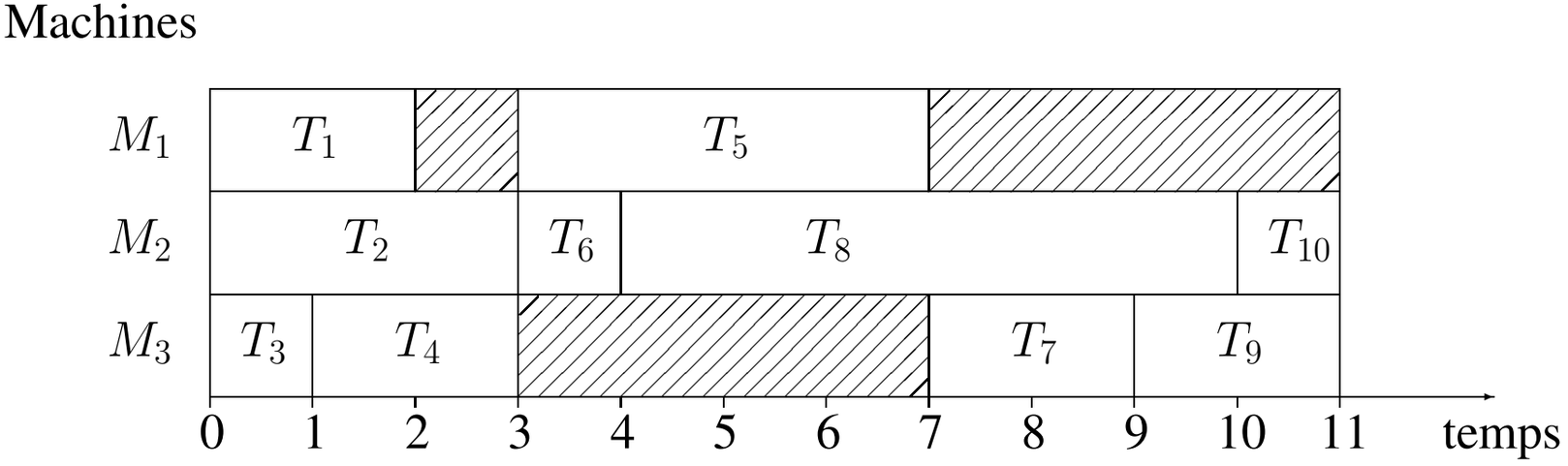
— A chaque instant $t \in [0, +\infty[$ (ordonnancement classique),
chaque machine est allouée à au plus une tâche (une machine est libre ou exécute exactement une tâche) et
chaque tâche est traitée par au plus une machine (une tâche est non affectée ou exécutée par exactement une machine).

— Chaque tâche T_i est traitée dans l'intervalle de temps $[r_i, +\infty[$. Une tâche est exécutée quand elle est disponible.

— Pour chaque pair (T_i, T_j) de tâches telles que $T_i < T_j$, le traitement de T_j ne commence que quand celui de T_i est achevé.

— Dans le cas d'ordonnancement non préemptif, aucune tâche n'est préemptée. Sinon le nombre de préemptions par tâche est fini.

Un ordonnancement pour l'instance précédente est donné à la figure ci-dessous.



2.4 Critères d'optimalité

Les paramètres suivants peuvent être calculés pour chaque tâche T_i, $i = 1, \ldots, n$; traitée dans un ordonnancement (admissible ou réalisable) donné :

— Une date d'achèvement (Completion time) c_i : c'est la date de fin de traitement de la tâche T_i.

— Une durée de flot ou de séjour (Flow time) f_i : c'est la somme du temps d'attente et du temps de traitement. C'est aussi la différence entre la date d'achèvement et la date d'arrivée. $f_i = c_i - r_i$.

— Une tardiveté, décalage temporel ou retard algébrique (lateness) l_i : c'est la différence entre la date d'achèvement et la date échue. $l_i = c_i - d_i$.

— Un retard (tardiness) t_i : $t_i = max\{l_i, 0\}$

— Un indicateur de retard u_i : $u_i = \begin{cases} 1 \; si \; c_i > d_i \\ 0 \; sinon \end{cases}$

Les différents paramètres de l'instance précédente sont :

T_i	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9	T_{10}
c_i	2	3	1	3	7	4	9	10	11	11
f_i	2	3	1	3	4	1	6	6	6	11
l_i	-1	-1	-2	1	0	-1	3	0	3	0
t_i	0	0	0	1	0	0	3	0	3	0
u_i	0	0	0	1	0	0	1	0	1	0

Pour évaluer des ordonnancements nous utilisons différents critères où on cherche à minimiser :

1. La longueur de l'ordonnancement (schedule length ou makespan) : $C_{max} = \max_{1 \leq i \leq n} \{c_i\}$
2. Le temps de flot moyen (mean flow time) : $\overline{F} = \frac{1}{n} \sum_{i=1}^{n} f_i$
3. La grande tardiveté (maximum lateness) : $L_{max} = \max_{1 \leq i \leq n} \{l_i\}$
4. Le nombre de tâches en retard (number of tardy jobs) : $N_T = \sum_{i=1}^{n} u_i$

Pour l'ordonnancement admissible de la figure précédente, nous avons :

— $C_{max} = max\{2, 3, 3, 2, 7, 4, 9, 10, 11, 11\} = 11$. Cet ordonnancement est optimal.

— $\overline{F} = (2 + 3 + 3 + 2 + 4 + 1 + 6 + 6 + 6 + 11)/10 = 43/10$. Cet ordonnancement n'est pas optimal.

— $L_{max} = max\{-1, -1, -2, 1, 0, -1, 3, 0, 3, 0\} = 3$. Cet ordonnancement est optimal.

— $N_T = 0 + 0 + 0 + 1 + 0 + 0 + 1 + 0 + 1 + 0 = 3$. Cet ordonnancement n'est pas optimal ($N_T\ optimal = 2$).

D'autres critères sont aussi utilisés :

1. Le temps de fin de traitement moyen (mean completion time) : $\overline{C} = \frac{1}{n}\sum_{i=1}^{n} c_i$
2. Le temps de flot moyen pondéré (mean weighted flow time) : $\overline{F_w} = \frac{\sum_{i=1}^{n} w_i f_i}{\sum_{i=1}^{n} w_i}$
3. Le temps de fin de traitement moyen pondéré (mean weighted completion time) : $\overline{C_w} = \left(\sum_{i=1}^{n} w_i c_i\right) / \left(\sum_{i=1}^{n} w_i\right)$
4. Le grand temps de flot (maximum flow time) : $F_{max} = \max_{1\leq i\leq n}\{f_i\}$
5. Le grand retard (maximum tardiness) : $T_{max} = \max_{1\leq i\leq n}\{t_i\}$
6. La tardiveté moyenne (mean lateness) : $\overline{L} = \frac{1}{n}\sum_{i=1}^{n} l_i$
7. La tardiveté moyenne pondérée (mean weighted lateness) : $\overline{L_w} = \left(\sum_{i=1}^{n} w_i l_i\right) / \left(\sum_{i=1}^{n} w_i\right)$
8. Le retard moyen (mean tardiness) : $\overline{T} = \frac{1}{n}\sum_{i=1}^{n} t_i$
9. Le retard moyen pondéré (mean weighted tardiness) : $\overline{T_w} = \left(\sum_{i=1}^{n} w_i t_i\right) / \left(\sum_{i=1}^{n} w_i\right)$
10. etc.

Remarque 1 *Quelques relations entre les différents critères :*

1. *Les critères $\overline{C_w}$, $\overline{F_w}$ et $\overline{L_w}$ sont équivalents et en particulier $\overline{C}$, $\overline{F}$ et $\overline{L}$.*
2. *En général, les critères C_{max}, F_{max} et L_{max} ne sont pas équivalents.*
3. *Un ordonnancement optimal pour L_{max} est aussi optimal pour T_{max}.*

Preuve : (Exercices)

3 Classification

Etant donné la diversité des problèmes de l'ordonnancement, un formalisme permettant de distinguer les différents problèmes de l'ordonnancement entre eux et de les classifier est utilisé. Ce formalisme comporte trois champs $\alpha|\beta|\gamma$ permettant de décrire les différentes entités d'un problème d'ordonnancement.

3.1 Champ α : ressources

Ce champ α décrit les machines utilisées : type de machines, nombre de machines et type d'atelier. Il est composé de deux sous-champs et désigné par $\alpha = \alpha_1\alpha_2$.

— $\alpha_1 \in \{1, P, Q, R, O, F, J\}$ décrit le type de machine.
  — $\alpha_1 = 1$: Une seule machine.
  — $\alpha_1 = P$: machines parallèles identiques.
  — $\alpha_1 = Q$: machines parallèles uniformes.
  — $\alpha_1 = R$: machines parallèles générales.
  — $\alpha_1 = O$: machines spécialisées (système open shop).
  — $\alpha_1 = F$: machines spécialisées (système flow shop).
  — $\alpha_1 = J$: machines spécialisées (système job shop).

— $\alpha_2 \in \{\emptyset, k\}$ dénote le nombre de machines pour $\alpha_1 \neq 1$.
  — $\alpha_2 = \emptyset$: le nombre de machines est variable.
  — $\alpha_2 = k$: le nombre de machines est égal à k (k est un entier positif non nul).

3.2 Champ β : contraintes

Ce champ β décrit les tâches et les contraintes liées à ces dernières. Il est composé de 5 sous-champs et désigné par $\beta = \beta_1, \beta_2, \beta_3, \beta_4, \beta_5$.

— $\beta_1 \in \{\emptyset, pmtn\}$: indique la possibilité de préemption des tâches.
  — $\beta_1 = \emptyset$: la préemption des tâches n'est pas autorisée.
  — $\beta_1 = pmtn$: la préemption des tâches est autorisée.

— $\beta_2 \in \{\emptyset, prec, tree, chains, ...\}$: indique le type de contraintes de précédence.
  — $\beta_2 = \emptyset$: pas de contraintes de précédence (les tâches sont indépendantes).
  — $\beta_2 = prec$: les contraintes de précédence sont quelconques.
  — $\beta_2 = tree$: les contraintes de précédence sont sous forme d'une arborecence.
  — $\beta_2 = chains$: les contraintes de précédence sont données sous forme de chaînes.
  — d'autres types de graphes orientés peuvent aussi être utilisés.

— $\beta_3 \in \{\emptyset, r_i\}$: décrit les dates de disponibilité.
  — $\beta_3 = \emptyset$: toutes les dates de disponibilité sont nulles.
  — $\beta_3 = r_i$: il existe des dates de disponibilité pour les tâches. Non toutes identiques

— $\beta_4 \in \{\emptyset, p_i = p, \ldots\}$.
  — $\beta_4 = \emptyset$: les temps de traitement des tâches sont quelconques et non tous identiques.
  — $\beta_4 = p_i = p$: les temps de traitement des tâches sont tous égaux à p (constants).

— $\beta_5 \in \{\emptyset, d_i, \tilde{d}_i\}$.
  — $\beta_5 = \emptyset$: il n'y a pas des dates échues pour les tâches ou il existe des dates échues pour les tâches dans le cas où le critère d'optimisation est en fonction de ces dernières.
  — $\beta_5 = d_i$: il existe des dates échues pour les tâches.
  — $\beta_5 = \tilde{d}_i$: il existe des dates limites pour les tâches.

Remarque 2 *La notation $\emptyset$ signifie qu'il ne faut mettre aucun symbole.*

3.3 Champ γ : critère d'optimalité

Le troisième champ γ décrit le critère d'optimalité. On cherche à minimiser γ avec : $\gamma \in \{C_{max}, \overline{C}, \overline{C_w}, T_{max}, \overline{T}, \overline{T_w}, L_{max}, \overline{U}, \overline{U_w}, -, \ldots\}$.

"$\gamma = -$" signifie qu'on cherche une solution réalisable.

3.4 Exemples

Considérons les problèmes suivants :

$P||C_{max}$: Il indique qu'on veut traiter un ensemble de tâches indépendantes et non interruptibles ayant des temps de traitement arbitraires et des dates de disponibilité nulles sur des machines parallèles identiques en temps minimal.

$O3|pmtn, r_i|\overline{F}$: Il indique qu'on veut traiter un ensemble de tâches indépendantes sur trois machines spécialisées dans un système open shop. Les opérations sont préemptibles et caractérisées par des temps de traitement et des dates de disponibilité arbitraires. L'objectif est la minimisation du temps de flot moyen.

$1|r_i, p_i = 1|L_{max}$: Il indique qu'on veut traiter un ensemble de tâches indépendantes et non interruptibles ayant des temps de traitement unitaires et des dates de disponibilité et des dates échues arbitraires sur une seule machine. L'objectif est la minimisation de la grande tardiveté.

$Q2|tree|C_{max}$: Il indique qu'on veut traiter un ensemble de tâches dépendantes, où les contraintes de précédences sont données sous forme d'un arborescence, et non interruptibles ayant des temps de traitement arbitraires sur deux machines parallèles uniformes. L'objectif est la minimisation de la date de fin de traitement.

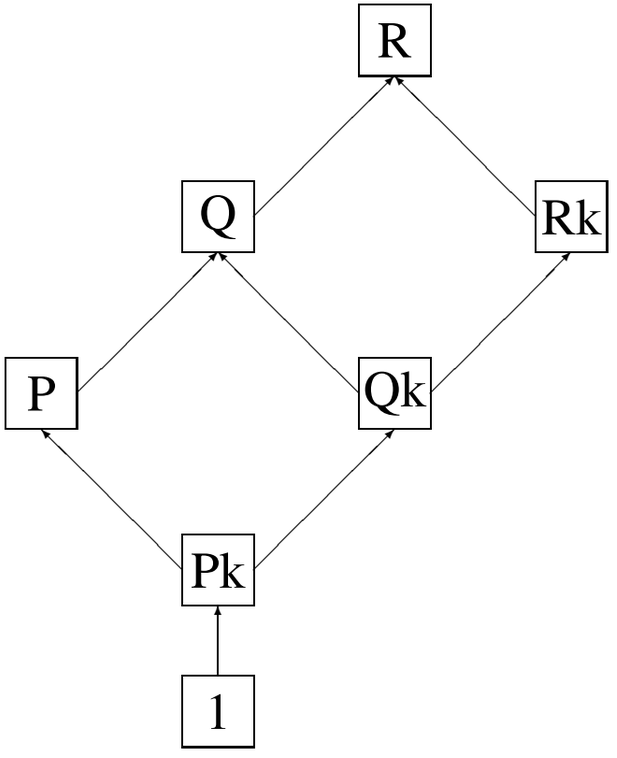
4 Réduction entre problèmes d'ordonnancement

Nous donnons ci-dessous quelques réductions (transformations polynomiales) entre problèmes d'ordonnancement.

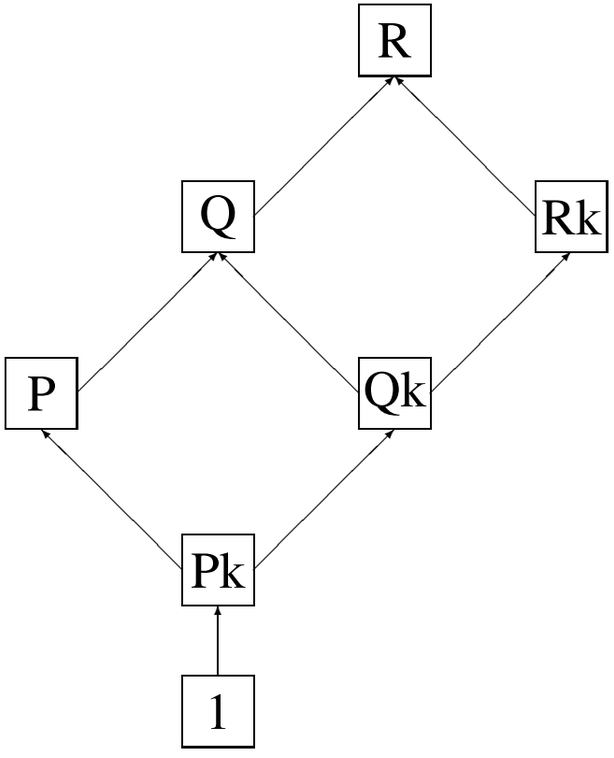
4.1 Machines

4.2 Contraintes sur les tâches

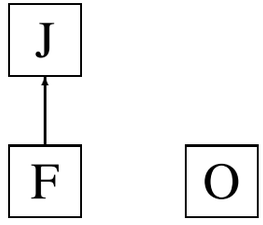
4.3 Critères



Machines parallèles



Machines parallèles



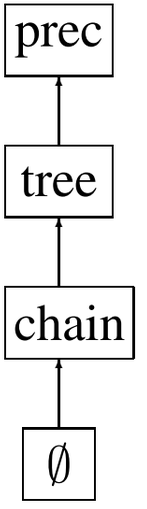
Machines spécialisées



Préemption



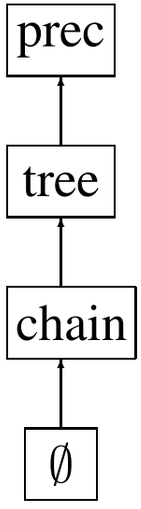
Préemption



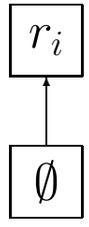
Contraintes de précédence



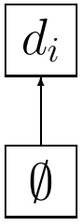
Préemption



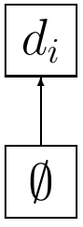
Contraintes de précédence



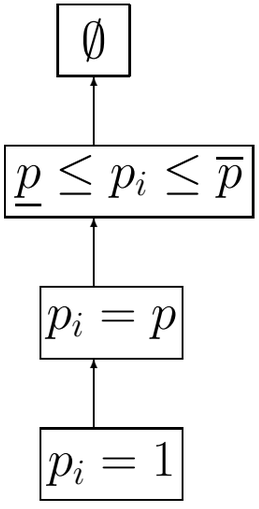
Dates de disponibilité



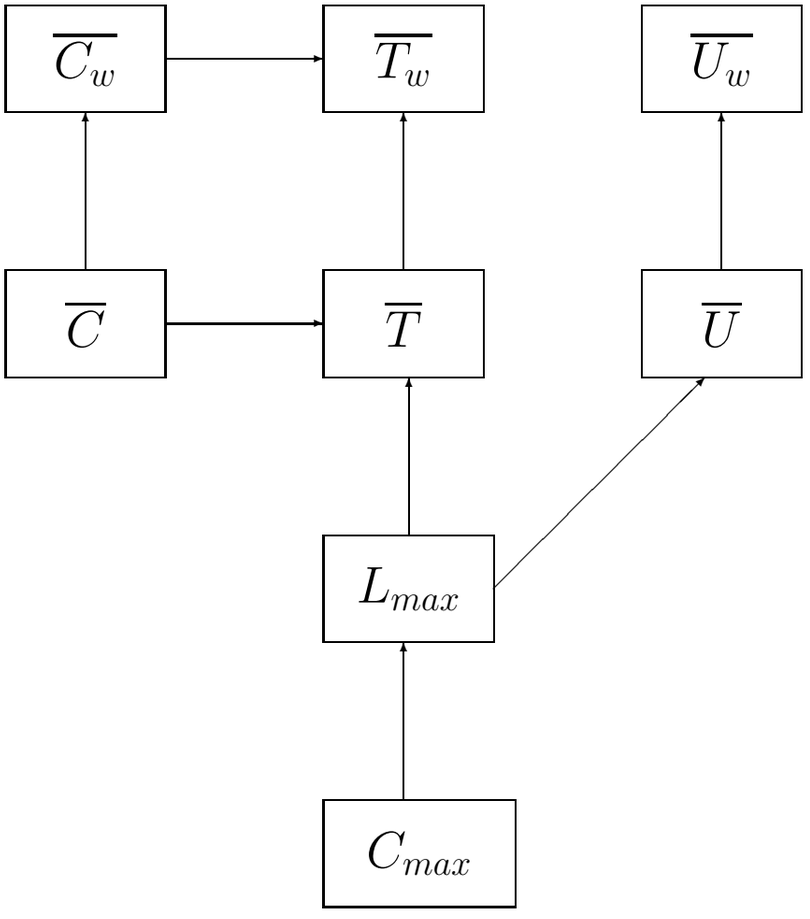
Dates échues



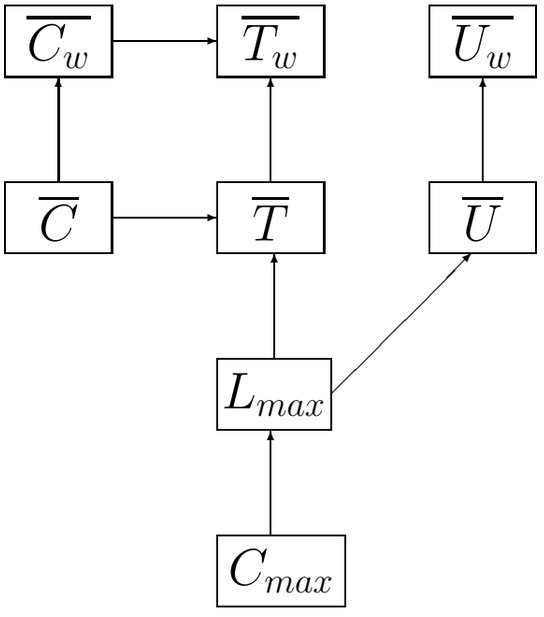
Dates échues



Temps de traitement


$\overline{C_w}$
$\overline{T_w}$
$\overline{U_w}$
$\overline{C}$
$\overline{T}$
$\overline{U}$
L_{max}
C_{max}

Critères



Critères

Nous avons :

— Σf_i se réduit à $\Sigma w_i f_i$ en posant $w_i = 1$ pour $i = 1, \ldots, n$.
Donc $\overline{C}$, $\overline{T}$ et $\overline{U}$ se reduisent respectivement à $\overline{C_w}$, $\overline{T_w}$ et $\overline{U_w}$.

— C_{max}, $\overline{C}$ et $\overline{C_w}$ se réduisent respectivement à L_{max}, $\overline{T}$ et $\overline{T_w}$ en posant $d_i = 0$ pour $i = 1, \ldots, n$.

— L_{max} se réduit à $\overline{T}$ et $\overline{U}$ (Exercice).